

RELAZIONE USO LLM NELLO SVILUPPO DI LILA

UNIVERSITÀ DEGLI STUDI DI SALERNO
CORSO: INGEGNERIA DEI LINGUAGGI
DI PROGRAMMAZIONE
2025/2026

Prof. Gennaro Costagliola

Mario Cosenza
NF22500094

1. Panorama degli Strumenti Generativi e Modelli LLM

Lo sviluppo di LILA è stato condotto attraverso un ecosistema eterogeneo di Intelligenza Artificiale, selezionando modelli specifici in base alla natura del task:

- **Modelli di Reasoning Avanzato (Fase di Design):** Per la definizione architetturale e la risoluzione di problemi logici complessi, è stato utilizzato **GPT-5.2 Thinking**. La sua capacità di "catena di pensiero" (**CoT**) si è rivelata fondamentale per mappare concetti astratti in strutture dati concrete prima ancora di scrivere codice.
- **Assistenti alla Codifica (Fase Implementativa):** Il "**workhorse**" quotidiano per la scrittura del codice è stato **Google Gemini 3 Pro**, utilizzato sia tramite l'interfaccia web per task isolati, sia integrato nell'IDE.
- **Embedded Pair Programmer:** L'ambiente di sviluppo **VS Code** è stato potenziato da **GitHub Copilot**, configurato per alternare l'uso dei modelli **GPT-5.1 Codex e Gemini 3 Pro** (per la gestione di contesti più ampi e refactoring complessi).

2. Casi d'Uso Specifici e Metodologia

L'integrazione dell'AI nel flusso di lavoro ha seguito un pattern ricorrente: "**Human-Defined Skeleton, AI-Enriched Implementation**". Ho definito le interfacce delle classi e la logica del grafo, delegando agli LLM il riempimento funzionale.

- **Sviluppo del Bridge MCP:** La creazione del bridge tra il compilatore Grammo (basato su output grezzi) e il formato strutturato (dict) richiesto dagli agenti è stata affidata a **GPT-5.2 Thinking**, sfruttando la sua capacità di ragionamento logico per mappare i dati. Successivamente, **Gemini 3 Pro** è stato cruciale per convertire questi script in server **FastMCP**, gestendo le complessità delle chiamate asincrone e la validazione **Pydantic**.
- **Refactoring e Tooling:** **GitHub Copilot** è stato impiegato massicciamente per la scrittura del client CLI, l'aggiunta di **Type Hints** (essenziali per LangGraph) e l'implementazione trasversale del logging.
- **Mitigazione delle Incompatibilità:** Un caso d'uso critico è stato lo sviluppo della classe `_GeminiSafeWrapper`. L'integrazione di **Gemma 3 27b IT** presentava problematiche nel binding dei tool rispetto allo standard **LangChain**. Utilizzando **Gemini 3 Pro**, è stato possibile scrivere un wrapper robusto per armonizzare le interfacce, permettendo l'uso agnostico dei modelli.
- **Testing e Parsing:** Le routine di cleaning delle stringhe e la logica di parsing dei risultati sono state accelerate da Copilot. Inoltre, la creazione dello script di benchmark per il calcolo della metrica **Pass@k** è stata interamente assistita, riducendo i tempi di setup dell'ambiente di valutazione.

3. Vantaggi e Svantaggi Riscontrati

L'adozione massiccia di strumenti generativi ha portato benefici tangibili ma ha anche evidenziato nuove categorie di rischi.

Vantaggi Operativi

1. **Abbattimento della Barriera d'Ingresso Tecnologica:** Tecnologie complesse come la gestione avanzata dei grafi in [LangGraph](#) hanno curve di apprendimento ripide. L'uso di Gemini come "tutor esperto" ha permesso di implementare protocolli avanzati senza dover studiare intere documentazioni, ma apprendendo attraverso esempi generati *ad hoc*.
2. **Focus sull'Architettura:** Delegando l'implementazione dei dettagli (regex, parsing JSON, chiamate HTTP) all'AI, ho potuto concentrare le energie sulla logica di orchestrazione degli agenti e sul design dei prompt, che costituiscono il vero valore aggiunto del progetto.
3. **Resilienza del Codice:** L'uso di Copilot per generare type hints ha elevato la qualità del codice base, riducendo i bug di runtime (**TypeError**) che sono frequenti in Python dinamico.

Svantaggi

Obsolescenza della Conoscenza (Library Hallucinations)

Si è riscontrato un problema critico con librerie in rapida evoluzione come [LangChain](#) e [LangGraph](#). I modelli, addestrati su dataset con cutoff temporali, tendevano a suggerire metodi deprecati (es. vecchie catene [LLMChain](#) invece delle nuove [RunnableSequence](#)). Questo ha richiesto una verifica costante sulla documentazione ufficiale, trasformando il lavoro di coding in un lavoro di "Fact Checking".

4. Lezioni Apprese e Commenti Personali

L'adozione di questi strumenti ha trasformato il mio ruolo da "**scrittore di codice**" a "**architetto e revisore**". La lezione fondamentale appresa è che **l'AI non può sostituire la visione architeturale**: tentare di far generare interi moduli da zero porta spesso a codice disconnesso. Al contrario, fornire uno scheletro solido e chiedere all'AI di "**riempire i vuoti**" o "**scrivere i test**" si è rivelata la strategia vincente.

Strumenti come *GPT-5.2 Thinking* eccellono nella logica pura, mentre *Copilot* è insostituibile per la produttività contestuale. Senza il supporto di questi assistenti, in particolare per la gestione delle complessità di *LangGraph* e *MCP*, i tempi di sviluppo di [LILA](#) sarebbero stati significativamente superiori.